NN

Fully Connected Layer

Forward Propagation: $z = Wx + b \rightarrow z_i = \sum_j W_{ij} x_j + b_i$

Local Derivative: $g_i = \dfrac{\partial L}{\partial z_i} \rightarrow \dfrac{\partial L}{\partial W_{ij}} = g_i x_j \quad \dfrac{\partial L}{\partial x_j} = \sum_i g_i W_{ij}$

Softmax + Cross-Entropy

$p_i = \dfrac{e^{z_i}}{\sum_{k=1}^{C} e^{z_k}}, \quad L = -\sum_{k=1}^{C} t_k \log p_k = -\log p_c, \quad \dfrac{\partial L}{\partial z_i} = p_i - t_i$

Sigmoid + Binary Cross-Entropy

$p = \dfrac{1}{1+e^{-z}}, \quad L = -[t \log p + (1-t)\log(1-p)], \quad \dfrac{\partial L}{\partial z} = p - t$

Activation Functions

Activation	Function	Derivative	Advantages	Disadvantages
Sigmoid	$\dfrac{1}{1+e^{-z}}$	$f(z)(1-f(z))$	Binary Classification	Vanishing Gradients
Tanh	$\dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$1-f(z)^2$	Zero-Centered	Vanishing Gradients
ReLU	$\max(0,z)$	$\begin{cases} 1 & z>0 \\ 0 & z<0 \end{cases}$	Simple	Dying ReLU
Leaky ReLU	$\begin{cases} z & z \geq 0 \\ \alpha z & z<0 \end{cases}$	$\begin{cases} 1 & z>0 \\ \alpha & z<0 \end{cases}$	Preserve Gradients	Hyperparameter α

Multi-Layer Perceptron

MLP: $z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \quad h^{(l)} = \phi_l(z^{(l)})$

Activation → Introduce Nonlinearity; Linear Function → Classification Logits

Multi-Layer → Improve Representation Efficiency; Perceptron → Final Linear Classification

Regression Losses

$L_1 = \dfrac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i| \rightarrow \dfrac{\partial L_1}{\partial \hat{y}_i} = \dfrac{1}{N}\operatorname{sign}(\hat{y}_i - y_i)$ → robust to outliers

$L_2 = \dfrac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \rightarrow \dfrac{\partial L_2}{\partial \hat{y}_i} = \dfrac{2}{N}(\hat{y}_i - y_i)$ → sensitive to outliers

Batch Normalization

Mean: $\mu_j = \dfrac{1}{N}\sum_i x_{ij}$, Variance: $v_j = \dfrac{1}{N}\sum_i (x_{ij} - \mu_j)^2$

Standardization: $\bar{x}_{ij} = \dfrac{x_{ij} - \mu_j}{\sqrt{v_j + \epsilon}}$; Affine Transform: $y_{ij} = \gamma_j \bar{x}_{ij} + \beta_j$

Normalize Intermediate Features → Stabilize Gradient Scales → Larger Learning Rates

Dropout

Ordinary Dropout: $y_i^{\text{train}} = m_i z_i, \quad y_i^{\text{test}} = q z_i$

Keep Probability q; Random Mask $m_i \sim \text{Bernoulli}(q)$

Deactivate Neurons → Reduce Co-Adaptation → Mitigate Overfitting / Better Generalization

AutoEncoder

Reconstruction Loss: $h = f_\theta(x), \quad \tilde{x} = g_\phi(h), \quad J = \dfrac{1}{2N}\sum_{i=1}^{N} \left\| x^{(i)} - \tilde{x}^{(i)} \right\|_2^2$

Low-Dimensional Bottleneck → Learn Latent Representation

Optimization

Mini-Batch SGD

Loss Function: $J_{\mathcal{B}_t}(\theta) = \dfrac{1}{|\mathcal{B}_t|}\sum_{i \in \mathcal{B}_t} \ell_i(\theta)$

Mini-batch Gradient: $g_t = \dfrac{1}{B_t}\sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \quad B_t = |\mathcal{B}_t|$

Gradient Descent: $\theta_{t+1} = \theta_t - \eta_t g_t, \quad g_t = \nabla J_{\mathcal{B}_t}(\theta_t)$

Taylor Expansion: $J_{\mathcal{B}_t}(\theta_t - \eta_t g_t) \approx J_{\mathcal{B}_t}(\theta_t) - \eta_t g_t^\top g_t = J_{\mathcal{B}_t}(\theta_t) - \eta_t \|g_t\|_2^2$

Smaller B_t → High Gradient Variance → Noisy Updates → Improve Generalization

Larger B_t → High Memory Usage → Fewer Updates per Epoch → Better Stability

Smaller η_t → Slow Convergence → More Epochs

Larger η_t → Repeated Overshooting → Oscillation / Divergence

Momentum

Exponential Moving Average: $v_t = \mu v_{t-1} + (1-\mu) g_t \quad \theta_{t+1} = \theta_t - \eta_t v_t$

Accumulate Past Gradients → Reinforce Consistent / Reduce Oscillation → Convergence

L2 Regularization

Regularization Term: $J_{\text{reg}}(\theta) = J_{\text{data}}(\theta) + \dfrac{\lambda}{2}\|\theta\|_2^2 \quad \nabla_\theta \dfrac{\lambda}{2}\|\theta\|_2^2 = \lambda\theta$

Shrinkage Factor: $\theta_{t+1} = \theta_t - \eta(g_t + \lambda\theta_t) = (1-\eta\lambda)\theta_t - \eta g_t$

Weight Decay → Penalize Large Weights → Reduce Overfitting → Improve Generalization

Resilient Propagation

Rprop: $\eta_{t,i} = \begin{cases} \min(\alpha\eta_{t-1,i}, \eta_{\max}), & g_{t,i}g_{t-1,i} > 0,\ \alpha > 1 \\ \max(\beta\eta_{t-1,i}, \eta_{\min}), & g_{t,i}g_{t-1,i} < 0,\ 0 < \beta < 1 \\ \eta_{t-1,i}, & g_{t,i}g_{t-1,i} = 0 \end{cases}$

$\theta_{t+1,i} = \theta_{t,i} - \eta_{t,i}\operatorname{sign}(g_{t,i})$

Consistent Signs → Increase Step Size → Faster Convergence

Opposite Signs → Decrease Step Size → Reduce Oscillation

Adaptive Learning Rate

AdaGrad: $s_{t,i} = s_{t-1,i} + g_{t,i}^2 \quad \theta_{t+1,i} = \theta_{t,i} - \dfrac{\eta}{\sqrt{s_{t,i}} + \epsilon} g_{t,i}$

RMSProp: $s_{t,i} = \rho s_{t-1,i} + (1-\rho) g_{t,i}^2 \quad \theta_{t+1,i} = \theta_{t,i} - \dfrac{\eta}{\sqrt{s_{t,i}} + \epsilon} g_{t,i}$

Measure Gradient Magnitudes → Adapt Learning Rates → Reduce Oscillation

Adaptive Moment Estimation

First-Moment Estimate: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ → Determine Update Direction

Second Raw-Moment Estimate: $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ → Adjust Update Step Size

Bias Correction: $\hat{m}_t = m_t / (1-\beta_1^t) \quad \hat{v}_t = v_t / (1-\beta_2^t)$ → Correct Zero Initialization

Adam: $\theta_{t+1} = \theta_t - \eta \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ → Adaptive Moment Estimation

AdamW: $\theta_{t+1} = (1-\eta\lambda)\theta_t - \eta \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ → Decoupled Weight Decay

Gradients Problems

Vanishing / Exploding Gradients → Repeated Multiplication of Local Derivatives

Gradient Clipping: $g_{\text{clip}} = g \cdot \min\left(1, \dfrac{c}{\|g\|_2}\right)$ → Bounded Gradient Norm

Weight Initialization

Xavier / Glorot: $W \sim \mathcal{N}\left(0, \dfrac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad W \sim U\left(-\sqrt{\dfrac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\dfrac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$ → sigmoid, tanh

Kaiming / He: $W \sim \mathcal{N}\left(0, \dfrac{2}{n_{\text{in}}}\right) \quad W \sim U\left(-\sqrt{\dfrac{6}{n_{\text{in}}}}, \sqrt{\dfrac{6}{n_{\text{in}}}}\right)$ → relu

Break Neuron Symmetry + Stabilize Signal Scale → Reduce Vanishing / Exploding

Data Augmentation

Rotation, Flipping, Cropping, Scaling, Jittering, Noise, Erasing

Learning Rate Schedules

Inverse-Time: $\eta_t = \dfrac{\eta_0}{1+dt}$, Step: $\eta_t = \eta_0 \gamma^{\lfloor t/K \rfloor}$, Exponential: $\eta_t = \eta_0 e^{-kt}$

Cosine Annealing: $\eta_t = \eta_{\min} + \dfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\dfrac{\pi t}{T}\right)$

Warmup: $\eta_t = \eta_{\max}\dfrac{t}{T_w}$

CNN

Advantages

Local Connectivity: Sparse Connections $\rightarrow$ Lower Computation / Local Feature Extraction

Weight Sharing: Reuse Filters $\rightarrow$ Fewer Parameters / Location-Independent Feature Detection

Translation Equivariance: $f(T_\Delta X) = T_\Delta f(X) \rightarrow$ Input Shifts, Feature Map Shifts Accordingly

Translation Invariance: $f(T_\Delta X) = f(X) \rightarrow$ Input Shifts, Classification Remains Unchanged

Forward Propagation

Feature Map: $y_{i,j,o} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} w_{o,c,u,v}\, x_{is_h-p_h+ud_h,\, js_w-p_w+vd_w,\, c}$

Kernel Size: $K_e = D(K-1)+1$, Max Index: $i_{max} + K - 1 \le L_{in} + 2p - 1$

Output Size: $L_{\text{out}} = \left\lfloor \dfrac{L_{\text{in}} + 2P - K_e}{S} \right\rfloor + 1$

Parameters=$(K_h K_w C_{in} + 1)\, C_{out}$, MACs=$H_{\text{out}} W_{\text{out}} C_{\text{out}} K_h K_w C_{\text{in}}$

Grouped: $C'_{\text{in}} = C_{\text{in}}/G$, Depthwise: $G = C_{\text{in}}$

Backward Propagation

Given $s = 1, p = 0, d = 1, c_{in} = 1, c_{out} = 1$

1D: $y_i = \sum_{k=0}^{K-1} w_k x_{i+k} + b \quad \delta_i = \dfrac{\partial J}{\partial y_i}$

$$\frac{\partial J}{\partial w_k} = \sum_i \delta_i x_{i+k} \quad \frac{\partial J}{\partial b} = \sum_i \delta_i \quad \frac{\partial J}{\partial x_a} = \sum_i \delta_i w_{a-i}$$

2D: $y_{i,j} = \sum_{u,v} w_{u,v} x_{i+u,j+v} + b \quad \delta_{i,j} = \dfrac{\partial J}{\partial y_{i,j}}$

$$\frac{\partial J}{\partial w_{u,v}} = \sum_{i,j} \delta_{i,j} x_{i+u,j+v} \quad \frac{\partial J}{\partial b} = \sum_{i,j} \delta_{i,j} \quad \frac{\partial J}{\partial x_{a,b}} = \sum_{i,j} \delta_{i,j} w_{a-i,b-j}$$

Receptive Field

$r_0 = 1 \quad j_0 = 1 \quad j_l = s_l j_{l-1} \quad r_l = r_{l-1} + (k_l - 1) d_l j_{l-1}$

$r_L = 1 + \sum_{l=1}^{L} (k_l - 1) d_l \prod_{m=1}^{l-1} s_m$

Pooling

Pooling	Formula	Advantages
Max	$y_{u,v,c} = \max_{(i,j)\in\Omega_{u,v}} x_{i,j,c}$	Salient Features $\rightarrow$ Improve Robustness
Average	$y_{u,v,c} = \dfrac{1}{\lvert\Omega_{u,v}\rvert} \sum_{(i,j)\in\Omega_{u,v}} x_{i,j,c}$	Smooth Features $\rightarrow$ Reduce Noise
Global Average	$y_c = \dfrac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} x_{i,j,c}$	Aggregate Features $\rightarrow$ Reduce Parameters

BatchNorm / Dropout

$$\mu_c = \frac{1}{M} \sum_{n,h,w} x_{n,h,w,c}, \; v_c = \frac{1}{M} \sum_{n,h,w} (x_{n,h,w,c} - \mu_c)^2$$

$$y_{n,h,w,c} = \gamma_c \frac{x_{n,h,w,c} - \mu_c}{\sqrt{v_c + \epsilon}} + \beta_c, \; y_{n,h,w,c} = \frac{m_{n,c}}{1-p} x_{n,h,w,c}$$

Mask-Aware Convolutions

Partial: $y_{i,j} = \begin{cases} \frac{K}{N_{i,j}} \text{Conv}_w (x \odot m)_{i,j} + b & N_{i,j} > 0 \\ 0 & N_{i,j} = 0 \end{cases} \rightarrow$ Valid Inputs Only

Submanifold Sparse: $y_{i,j} = \begin{cases} \text{Conv}_w (x \odot m)_{i,j} + b & m_{i,j} = 1 \\ 0 & m_{i,j} = 0 \end{cases} \rightarrow$ Active Inputs Only

Modern Convolutions

Model	Key Idea	Advantages
VGG	Stacked Small Convolutional Kernels	Expand Receptive Field Using Less Parameters
NiN	Pointwise Convolution for Channel Mixing	Fuse Features Across Channels Instead of Fully Connected Layers
GoogLeNet	Multi-Scale Convolutions	Concatenate Feature Map to Capture Multi-Scale Features
ResNet	Residual Skip Connections	Improve Gradient Flow & Enable Deeper Networks
DenseNet	Dense Connections	Enhance Feature Reuse and Improve Gradient Flow
ResNeXt	Grouped Convolutions	Improve Representation with Efficient Computation
MobileNet	Depthwise Separable Convolutions	Reduce Parameters and Reduce Computation
EfficientNet	Compound Scaling	Improve Accuracy–Efficiency Trade-off

RNN

Sequence Probability Factorization

Probability Chain Rule: $P(x_{1:T}) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{1:t-1})$

First-Order Markov Assumption: $P(x_{1:T}) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})$

Second-Order Markov Assumption: $P(x_{1:T}) = P(x_1, x_2) \prod_{t=3}^{T} P(x_t \mid x_{t-2}, x_{t-1})$

Hidden Markov Model: $P(x_{1:T}, h_{1:T} \mid \theta) = P(h_1) \prod_{t=2}^{T} P(h_t \mid h_{t-1}) \prod_{t=1}^{T} P(x_t \mid h_t)$

Transition Distribution: $P(h_t \mid h_{1:t-1}) = P(h_t \mid h_{t-1})$

Emission Distribution: $P(x_t \mid h_{1:t}, x_{1:t-1}) = P(x_t \mid h_t)$

Vanilla Recurrent Neural Network

Forward Recurrence: $a_t = W_x x_t + W_h h_{t-1} + b_h$, $h_t = \tanh(a_t)$, $z_t = W_z h_t + b_z$, $p_t = \text{softmax}(z_t)$

Parameter Count: $H(I + H + 1) + O(H + 1)$

All RNN parameter counts assume one unidirectional layer, one combined bias per gate, and the output layer; I, H, O are input, hidden and output dimensions.

Backward Propagation Through Time

$\ell_t = -\sum_{c=1}^{C} y_{t,c} \log p_{t,c}$, $L = \sum_{t=1}^{T} \ell_t$

$$\delta_t = \frac{\partial L}{\partial a_t} = \left(\frac{\partial h_t}{\partial a_t}\right)^\top \frac{\partial L}{\partial h_t}$$

$$\frac{\partial L}{\partial h_t} = \left(\frac{\partial z_t}{\partial h_t}\right)^\top \frac{\partial L}{\partial z_t} + \left(\frac{\partial a_{t+1}}{\partial h_t}\right)^\top \frac{\partial L}{\partial a_{t+1}}$$

$\delta_t = \left(W_z^\top (p_t - y_t) + W_h^\top \delta_{t+1}\right) \odot (1 - h_t^2), \quad \delta_{T+1} = 0$

$\nabla_{W_z} L = \sum_{t=1}^{T} (p_t - y_t) h_t^\top$, $\nabla_{W_x} L = \sum_{t=1}^{T} \delta_t x_t^\top$, $\nabla_{W_h} L = \sum_{t=1}^{T} \delta_t h_{t-1}^\top$

$\nabla_{b_z} L = \sum_{t=1}^{T} (p_t - y_t)$, $\nabla_{b_h} L = \sum_{t=1}^{T} \delta_t$

$J_t = \dfrac{\partial h_t}{\partial h_{t-1}} \rightarrow \dfrac{\partial h_T}{\partial h_t} = J_T J_{T-1} \cdots J_{t+1}$: Repeated Jacobian Multiplication $\rightarrow$ Vanishing / Exploding Gradients

Long Short-Term Memory

Forget Gate: $f_t = \sigma(W_f [x_t; h_{t-1}] + b_f) \rightarrow$ control how much Old Memory to Retain

Input Gate: $i_t = \sigma(W_i [x_t; h_{t-1}] + b_i) \rightarrow$ control how much New Information to Write

Output Gate: $o_t = \sigma(W_o [x_t; h_{t-1}] + b_o) \rightarrow$ control how much New Memory to Expose

Candidate Cell State: $\tilde{c}_t = \tanh(W_c [x_t; h_{t-1}] + b_c) \rightarrow$ compute the New Information

Cell State Update: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \rightarrow$ compute the New Memory

Hidden State: $h_t = o_t \odot \tanh(c_t) \rightarrow$ produce the Representation

$\left.\dfrac{\partial c_t}{\partial c_{t-1}}\right|_{\text{direct}} = \text{diag}(f_t) \rightarrow \left.\dfrac{\partial c_T}{\partial c_t}\right|_{\text{direct}} = \prod_{k=t+1}^{T} \text{diag}(f_k)$: Separate Memory + Additive Update $\rightarrow$ Better Gradients Flow

Parameter Count: $4H(I + H + 1) + O(H + 1)$

GRU

Reset Gate: $r_t = \sigma(W_r [x_t; h_{t-1}] + b_r) \rightarrow$ control how much History to Reuse

Update Gate: $z_t = \sigma(W_z [x_t; h_{t-1}] + b_z) \rightarrow$ control how much Old Memory to Retain

Candidate Hidden State: $\tilde{h}_t = \tanh(W_x x_t + r_t \odot (W_h h_{t-1}) + b_h) \rightarrow$ compute the New Candidate

Hidden State Update: $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \rightarrow$ produce the Representation

$\left.\dfrac{\partial h_t}{\partial h_{t-1}}\right|_{\text{direct}} = \text{diag}(z_t) \rightarrow \left.\dfrac{\partial h_T}{\partial h_t}\right|_{\text{direct}} = \prod_{k=t+1}^{T} \text{diag}(z_k)$: Unified Memory + Additive Update $\rightarrow$ Better Gradient Flow

Parameter Count: $3H(I + H + 1) + O(H + 1)$

Autoregressive Generation

$P(x_{1:T}) = \prod_{t=1}^{T} P(x_t \mid x_{<t}) \approx \prod_{t=1}^{T} P_\theta(x_t \mid h_{t-1}) = \prod_{t=1}^{T} p_{t-1}[x_t]$

$L = -\log P(x_{1:T}) = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t}) \approx -\sum_{t=1}^{T} \log P_\theta(x_t \mid h_{t-1}) = -\sum_{t=1}^{T} \log p_{t-1}[x_t]$

Seq2Seq

Encoder: $v = \text{RNN}(x_1, \ldots, x_T) = h_T^{\text{enc}}$

Decoder: $h_0^{\text{dec}} = v = h_T^{\text{enc}}$,
$P_\theta(y_{1:T'} \mid x_{1:T}) = \prod_{t=1}^{T'} P_\theta(y_t \mid v, y_{<t}) = \prod_{t=1}^{T'} P_\theta(y_t \mid h_t^{\text{dec}}) = \prod_{t=1}^{T'} p_t[y_t]$